

Mastering Java CompletableFuture

Part 1: The Foundation

Why CompletableFuture exists, when to use it, and the mental model that will save you hours of debugging.



Morty

Follow

5 min read · Feb 4, 2026

The Problem: Why Blocking I/O Kills Your Application

Picture this: your Spring Boot service handles 100 requests per second. Each request fetches user data from a database, a 100ms operation. With a typical thread pool of 200 threads, you're already at capacity. One slow query, and your entire service starts queuing requests.

This is the classic **thread-per-request bottleneck**. Your threads spend 90% of their time *waiting*, waiting for the database, waiting for an HTTP response, waiting for a file to load. They're not computing anything; they're just... blocked.

`CompletableFuture` exists to solve this problem.

The Evolution: From Thread to CompletableFuture

Let's trace how Java's concurrency model evolved, because understanding this history helps you understand *when* to use each tool.

Level 1: Raw Threads (Java 1.0)

```
// The ancient way
Thread thread = new Thread(() -> {
    User user = database.fetchUser(userId);
    System.out.println(user);
});
thread.start();
// How do I get the result back?
```

Problem: No way to return a value. No composition. Manual thread management.

Level 2: ExecutorService + Future (Java 5)

```
ExecutorService executor = Executors.newFixedThreadPool(10);
Future<User> future = executor.submit(() -> database.fetchUser(userId));

// To get the result, you MUST block:
User user = future.get(); // Blocks until done
```

Problem: `Future.get()` blocks. You can't chain operations. You can't combine futures elegantly.

Level 3: CompletableFuture (Java 8)

```
CompletableFuture<User> future = CompletableFuture.supplyAsync(  
    () -> database.fetchUser(userId)  
);  
  
// Non-blocking transformation!  
future.thenApply(user -> user.getEmail())  
    .thenAccept(email -> sendWelcomeEmail(email));  
// Thread is FREE to do other work
```

The breakthrough: Callbacks, composition, and non-blocking operations.

Level 4: @Async (Spring)

```
@Async  
public CompletableFuture<User> fetchUser(Long userId) {  
    return CompletableFuture.completedFuture(database.fetchUser(userId));  
}
```

Sounds convenient, but... it has subtle pitfalls we'll cover in Part 3. Spoiler: I recommend manual `CompletableFuture` over `@Async` in most cases.

The Mental Model: Think “Promise”, Not “Thread”

If you've worked with JavaScript, `CompletableFuture` is Java's `Promise`. It represents a value that will be available in the future.

```
CompletableFuture<User> = "I promise to give you a User... eventually."
```

The key insight: **you don't wait for the promise to resolve**. Instead, you tell it *what to do* when the value arrives:

```
fetchUserAsync(userId)
    .thenApply(user -> user.getEmail())    // "When you have the user, extract
    .thenAccept(email -> log.info(email));  // "When you have the email, log it
```

The thread that calls this code **returns immediately**. It doesn't block. It doesn't wait. It's free to handle other requests.

Your First CompletableFuture: supplyAsync vs runAsync

There are two ways to start an async operation:

supplyAsync — When you need a result

```
// Returns CompletableFuture<User>
CompletableFuture<User> future = CompletableFuture.supplyAsync(() -> {
    return database.fetchUser(userId); // Returns something
});
```

runAsync — Fire-and-forget (no return value)

```
// Returns CompletableFuture<Void>
CompletableFuture<Void> future = CompletableFuture.runAsync(() -> {
    auditLog.record("User viewed page"); // Side effect only
});
```

Rule of thumb: Use `supplyAsync` 90% of the time. You almost always want the result.

The Cardinal Sin: Blocking Your Async Code

Here's the most common mistake I see in production code:

```
// ❌ WRONG: This defeats the entire purpose of async
public User getUser(Long userId) {
    CompletableFuture<User> future = fetchUserAsync(userId);
    return future.get(); // BLOCKS! Thread sits idle.
}
```

You've created an async operation... then immediately blocked waiting for it. You might as well have called the database synchronously. In fact, it's *worse*, you added overhead for no benefit.

The correct approach: Return the future

```
// ✅ CORRECT: Let the caller decide
public CompletableFuture<User> getUser(Long userId) {
    return fetchUserAsync(userId);
    // Caller can chain operations without blocking
}
```

Now the caller can do:

```
userService.getUser(userId)
    .thenApply(this::enrichProfile)
    .thenAccept(this::cacheResult);
// Thread returns IMMEDIATELY
```

Transforming Results: thenApply and thenAccept

Once you have a `CompletableFuture`, you'll want to transform or consume its result.

thenApply, Transform the value (like Stream.map)

```
CompletableFuture<String> emailFuture = fetchUserAsync(userId)
    .thenApply(user -> user.getEmail()); // User → String
```

thenAccept, Consume the value (side effects)

```
fetchUserAsync(userId)
    .thenAccept(user -> {
        log.info("Fetched user: {}", user.getUsername());
        metrics.increment("user.fetch.success");
    });
```

Chain them together

```
fetchUserAsync(userId)
    .thenApply(user -> user.getEmail())           // User → String
    .thenApply(email -> email.toUpperCase())       // String → String
    .thenAccept(email -> sendEmail(email));       // Consume
```

Each step is **non-blocking**. The thread that starts this chain returns immediately.

completedFuture: The Shortcut for Cached Values

Sometimes you already have the value. Maybe it's cached, or validation failed early. You don't need an async operation, but your API returns `CompletableFuture`.

```
public CompletableFuture<User> fetchUser(Long userId, User cachedUser) {
    if (cachedUser != null) {
        // Return immediately, no async operation needed
        return CompletableFuture.completedFuture(cachedUser);
    }
}
```

```
    return fetchUserAsync(userId);  
}
```

This is also **invaluable for testing**, we'll cover that in Part 4.

join() vs get(): When You Must Block

Sometimes blocking is unavoidable:

- At the edge of your application (e.g., a synchronous controller)
- In tests
- In `main()` methods

When you must block, prefer `join()` over `get()`:

```
// get() throws checked exceptions – annoying to handle  
try {  
    User user = future.get();  
} catch (InterruptedException | ExecutionException e) {  
    // Ugly exception handling  
}  
  
// join() throws unchecked CompletionException – cleaner  
User user = future.join(); // Much cleaner
```

But remember: every `join()` or `get()` is a code smell. Ask yourself: "Can I return the future instead?"

When NOT to Use CompletableFuture

CompletableFuture isn't always the answer. Skip it when:

1. CPU-bound operations

```
// ❌ Don't do this
CompletableFuture.supplyAsync(() -> fibonacci(1000000));

// ✅ Use parallel streams instead
list.parallelStream().map(this::compute).collect(toList());
```

CompletableFuture shines for I/O-bound work (waiting). For CPU-bound work (computing), parallel streams are more efficient.

2. Simple synchronous operations

```
// ❌ Over-engineering
CompletableFuture.supplyAsync(() -> user.getName().toUpperCase());

// ✅ Just do it synchronously
user.getName().toUpperCase();
```

If the operation takes microseconds, the async overhead isn't worth it.

3. When strict ordering is required

If operations *must* happen in sequence with no parallelism benefit, async adds complexity without value.

Quick Reference: The Basics

Method	Use When	Returns
<code>supplyAsync(supplier)</code>	Starting async computation with result	<code>CompletableFuture<T></code>
<code>runAsync(runnable)</code>	Fire-and-forget (no result)	<code>CompletableFuture<Void></code>
<code>thenApply(fn)</code>	Transform value (like <code>map</code>)	<code>CompletableFuture<U></code>
<code>thenAccept(consumer)</code>	Consume value (side effects)	<code>CompletableFuture<Void></code>
<code>completedFuture(value)</code>	Return cached/immediate value	<code>CompletableFuture<T></code>
<code>join()</code>	Block and get result (unchecked)	<code>T</code>

Key Takeaways

1. **CompletableFuture solves thread blocking** — your threads can do other work while waiting for I/O.
2. **Never call `get()` or `join()` immediately** — return the future and let callers chain operations.
3. **Think in callbacks** — “when this completes, do that” instead of “wait for this, then do that.”
4. **Use `supplyAsync` for computations with results**, `runAsync` for fire-and-forget.

5. `completedFuture` is your friend for caching, testing, and early returns.

Coming Up: Part 2 , Composition

In Part 1, we learned the basics. But the real power of `CompletableFuture` is **composition** — combining multiple async operations elegantly.

In Part 2, we'll cover:

- The critical difference between `thenApply` and `thenCompose`
- Running operations in parallel with `allOf` and `thenCombine`
- Building a real-world dashboard that loads user, orders, and payments concurrently
- Why you should never use `ForkJoinPool.commonPool()` in production

See you there.

Check out the GitHub repo for <https://github.com/mo0rti/Learning-Completable-Future>

Part 2: Composition — The Real Power →

Spring Boot

Java

Concurrency

Asynchronous Programming

CompletableFuture



Written by Morty

648 followers · 40 following

Follow

A Software Engineer busy creating top-notch apps. <https://mortitech.com/>

No responses yet

